

# **Draft: A Review of Type Theories: Martin L  f, Homotopy and Cubical**

A Short Review

Naveen Maurya

April 12, 2026

# 1 Introduction

*Type Theory*, and in particular, its variants such as Homotopy Type Theory, Cubical Type Theory etc. are foundational systems for mathematics, alternative to ZF or other set theories. However, as opposed to any of the usual set-theoretic foundations, which consists of a number of axioms, defining sets and how they behave, in the language of first-order logic, which is separately assumed to be established beforehand.

## 2 A Brief Description of Type Theory

We shall describe a standard version of type theory called Martin-Löf Type Theory (MLTT). Set theoretic foundations usually consist of a number of *axioms*, written in the language of first-order logic, which is separately assumed beforehand. The logic's deductive system dictates how inferences can be made (how the logic behaves), while the *axioms* only define and describe sets and how they behave. This forms a basis to build mathematics, using sets to describe mathematical objects. However, MLTT combines both of these together into the deductive system itself. The deductive system consists of the primitive notion of 'types', the building blocks for mathematical objects here, and specifies by itself how they ought to behave.

MLTT consists of *rules* for deriving *judgements*, which are either of the form  $a : A$  meaning ' $a$  is a term/instance of type  $A$ ', or  $a \equiv b : A$  meaning ' $a$  is judgementally equal to  $b$ '. Judgemental equality is a strict notion of equality which classifies objects which are definitionally the same within the intrinsic language of the theory. A separate notion 'propositional equality', denoted with  $=$ , is a weaker but more faithful to the original notion of equality of mathematical objects.

The language of MLTT is an extension of dependently-typed  $\lambda$ -calculus, with variables  $x_1, x_2, \dots$ , primitive constants  $c, c', c'', \dots$ , and defined constants  $f, f', f'', \dots$ , with terms  $t, t', t'', \dots$  given by the grammar

$$t \rightarrow x \mid \lambda x. t \mid t(t') \mid c \mid f$$

### 2.1 Universes

Type Theory 'postulates' a sequence of cumulative universes à la Russell,  $\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \dots$  with

- $\mathcal{U}_i : \mathcal{U}_{i+1}$  for each  $i$
- If  $A : \mathcal{U}_i$ , then  $A : \mathcal{U}_{i+1}$

Any type in the theory is an instance of some universe in this chain. Each universe is also a type, and an instance of universes above it in this hierarchy. But unless necessary, we omit the index from the universe and write  $A : \mathcal{U}$ .

**Remark.** This sequence, while indexed by natural numbers, is not related to the Natural Numbers that will be defined in the theory. The indices are coming from the metatheory. We shall see, the arithmetic object of Natural Numbers ( $\mathbb{N}$ ) is richer and needs more information to be properly defined.

### 2.2 Logic

### 2.3 Type Formers and Inductive Types

The rest of the deductive rules of Type Theory go on to give schemas to construct, manipulate, and eliminate various constructions (known as type formers/inductive types). A number of these constructions are *synthetically* baked into the deductive system, known as *type formers*. Type Theories also allow the user to define their own constructions, known as inductive types, requiring the same criteria to be defined.

The general scheme for these type formers/inductive types is as follows. A type former/inductive type is defined by:

- A list of **formation rules**, describing how the inductive type can be introduced into context.
- A list of **constructors/introduction rules**, to introduce new terms of these types.
- **Elimination rules** to give a recipe how to use/consume elements of the type.  
(Usually, this involves a *recursion principle* to define functions on the type, and a stronger *induction principle* to define dependent functions on the type.)
- A **computation rule**, that tells how the elimination acts on the constructors.

Here is a brief, informal description of these type formers in Type Theory. A detailed, formal description is available in Appendix I [1].

## 2.4 Custom Inductive Types

Type Theories provide a schema to build newer, user-defined inductive types as well. Conceptually, an inductive type is a type *freely generated* by a number of constructors, which may involve functions that output the given type by (possibly) taking as input terms of the same type. This intuition needs to be formalized carefully so as to ensure that only meaningful inductive types may be constructed.

**Remark.** *Inductive types are objects one can perform induction on! The induction principle from Natural Numbers can be extended more generally, To prove a statement about all of  $\mathbb{N}$ , one simply needs to prove it for 0 and given it is true for  $n$ , prove it for  $\text{succ}(n)$ .*

*Similarly, the induction principle for a given inductive type is the statement that to prove a statement for all its instances, it suffices to prove it in a few special cases, built on the constructors of the type.*

### 2.4.1 W-Types

There is a complete formalism of inductive types upto certain demands for the constructors.

**Definition 1** (W-Types). *Given  $A : \mathcal{U}, B : A \rightarrow \mathcal{U}$ , the W-Type  $\mathbf{W}_{(a:A)}B(a)$  is defined by the constructor*

$$\text{sup} : \left( \prod_{(a:A)} B(a) \rightarrow \mathbf{W}_{(a:A)}B(a) \right) \rightarrow \mathbf{W}_{(x:A)}B(x)$$

*and the induction principle, to prove  $E : \mathbf{W}_{(a:A)}B(a) \rightarrow \mathcal{U}$  for all elements of  $\mathbf{W}_{(a:A)}B(a)$ , it suffices to prove it for  $\text{sup}(a, f)$  for each  $a : A$ , and  $f : (B(a) \rightarrow \mathbf{W}_{(a:A)}B(a))$ , assuming it holds for all  $f(b)$  with  $b : B(a)$ .*

A W-Type  $\mathbf{W}_{(a:A)}B(a)$  may be interpreted as the type of well-founded trees with  $A$ -many ways to branch out from each node, corresponding to a constructor. Each  $a : A$  corresponds to a constructor, each of which take  $B(a)$ -many instances of  $\mathbf{W}_{(a:A)}B(a)$  as branch nodes. The type  $\mathbf{W}_{(a:A)}B(a)$  is the type of objects *freely generated* by these constructors. If an inductive type may be expressed equivalently as generated by  $\text{sup}$  as described, it can be formalized as a W-Type.

- The boolean type  $\mathbf{2}$  is equivalently expressed as the W-Type  $\mathbf{W}_{(a:A)}B(a)$  where  $A \equiv \mathbf{2}$  with  $B(0_2) \equiv \mathbf{0}$  and  $B(1_2) \equiv \mathbf{0}$ , that is generated by two constructors,  $0_2 : \mathbf{2}$  and  $1_2 : \mathbf{2}$ .
- The natural numbers type  $\mathbb{N}$  is equivalent to  $\mathbf{W}_{(a:A)}B(a)$  where  $A \equiv \mathbf{2}$ , with  $B(0_2) \equiv \mathbf{0}$  and  $B(1_2) \equiv \mathbf{1}$ , i.e., generated by two constructors, corresponding to  $\text{zero} : \mathbb{N}$ , and  $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$ .
- The type  $\mathbf{W}_{(x:\mathbf{1}+A)}B(x)$  with  $B(\text{inl}(\star)) \equiv \mathbf{0}$  and  $B(\text{inr}(a)) \equiv \mathbf{1}$ , corresponds to the type  $\text{List}(A)$  of finite length strings over the alphabet  $A$ , i.e. generated by  $\text{nil} : \text{List}(A)$  and  $\text{cons} : A \rightarrow \text{List}(A) \rightarrow \text{List}(A)$ .

## 3 Homotopy Type Theory

Homotopy Type Theory (HoTT) extends MLTT by introducing a topological/homotopical interpretation of types, an interpretation of

### 3.1 A Homotopical Interpretation

In HoTT, types can be interpreted as either topologically as spaces, or as  $\infty$ -groupoids from higher-dimensional category theory. Any topological space can be classified upto homotopy equivalence by considering its *fundamental  $\infty$ -groupoid*. This follows from the fundamental  $\infty$ -groupoid construction being adjoint to the geometric realization functor. (This is the *homotopy hypothesis/theorem*).[Pursuing Stacks]

### 3.2 Identity Types

The precise manner in which types correspond to spaces or  $\infty$ -groupoids is demonstrated by constructing an associated  $\infty$ -groupoid.

In the homotopy interpretation, the identity type  $\text{Id}_A(x, y)$  corresponds to the space of paths from  $a : A$  to  $b : B$  endowed with the compact-open topology.  $\text{refl}_x$  is the constant path at  $x$ .

**Proposition 1.** *Given  $x, y, z : A$ , there exist the following maps*

- (i)  $(x = y) \rightarrow (y = x)$
- (ii)  $(x = y) \rightarrow (y = z) \rightarrow (x = z)$

*Proof.* (i) By path induction, assume  $x \equiv y$  and  $p \equiv \text{refl}_x$ . Now simply exhibit  $\text{id}_{x=A} \equiv (\lambda a.a)$ . (ii) By path induction, assume  $x \equiv y$  and  $p \equiv \text{refl}_x$ . Exhibit  $(\lambda a.b.b)$ .  $\square$

Denote the map in (i) as  $(-)^{-1}$ , and (ii) as  $(-\cdot-)$ . Then we have the following results.

**Proposition 2.** *Let  $x, y, z, w : A$ ,  $p : x =_A y$ ,  $q : y =_A z$  and  $r : z =_A w$ . Then we have:*

- (i)  $p = \text{refl}_x \cdot p$ , and  $p = p \cdot \text{refl}_y$
- (ii)  $p \cdot p^{-1} = \text{refl}_x$  and  $p^{-1} \cdot p = \text{refl}_y$
- (iii)  $(p \cdot q) \cdot r = p \cdot (q \cdot r)$

*Proof.* Follows from path induction. □

A remarkable feature is that while (i) and (ii) can be proven judgementally, (iii) is only true propositionally, i.e., upto a path in the higher path space. Strikingly, we also have all the required higher coherences propositionally using only path induction. This gives us the (weak)  $\infty$ -groupoid interpretation.

### 3.3 Axioms of Function Extensionality and Univalence

Some expected semantic behaviour does not directly follow from the machinery of HoTT laid out so far. To enforce such behaviour we introduce a few ‘axioms’ into our theory, which are merely inhabitants of some type without an inductive construction.

The notion of equality for functions is too restrictive to capture their expected behaviour. Classically, we expect two functions to be equal if they take the same values on all the inputs. To ensure semantic well-behavedness, we must enforce this as an axiom.

**Axiom 1** (Function Extensionality). *Let  $f, g : \prod_{(x:A)} B(x)$ , then we have*

$$(f = g) \simeq \left( \prod_{(x:A)} f(x) =_{B(x)} g(x) \right)$$

In, Voevodsky proposed the univalence axiom,

**Axiom 2** (Univalence). *For  $A, B : \mathcal{U}$ , there is an equivalence:*

$$(A =_{\mathcal{U}} B) \simeq (A \simeq B)$$

In particular, if  $A \simeq B$ , then  $A =_{\mathcal{U}} B$ . Roughly speaking, this means “isomorphic objects are equal”. In a univalent universe  $\mathcal{U}$ , isomorphic objects/equivalent types can be treated as equal and one can transport (theorems/proofs/properties) across them.

### 3.4 Higher Inductive Types

Following this interpretation, HoTT allows us to extend our domain of inductive definitions, from standard  $W$ -types which are types generated freely by its constructors, to types generated by constructors along with relations.

We allow the constructors to also be higher-dimensional paths between points. In this manner, free generation is equivalent to generation on points quotiented by relations (corresponding to these higher paths).

Such a *higher inductive type* hardcoded in HoTT is the circle type  $\mathbb{S}^1$  generated by the constructors

- A point `base` :  $\mathbb{S}^1$
- A path `loop` : `base`  $=_{\mathbb{S}^1}$  `base`

This inductive type corresponds to a chosen basepoint `base`, and a nontrivial path `loop` connected the basepoint, giving topologically a circle.

One can build many of the familiar mathematical objects such as the integers, higher spheres, CW-complexes etc. using this general schema. Various approaches exist to formalize such a schema, but have only yielded partial satisfaction. Some of the existing approaches are using Quotient Inductive Types  $\square$ , Homotopy Initial Algebras  $\square$ , and computationally most satisfactory, Cubical Type Theory  $\square$ . A satisfactory formalism remains an open problem.

## 4 Cubical Type Theory